

Technically, a **reverse proxy** is a component that sits *between* a client and a server to intercept a request **before** it reaches the backend. However, you can use a reverse proxy to perform actions **after** the backend has generated a response but **before** that response is delivered back to the client.  Ngrok +3

How a Reverse Proxy Handles a Post-Call Response

In a standard workflow, the reverse proxy does not just forward requests; it also manages the "return leg" of the communication.  miniOrange +2

- **Response Modification:** Reverse proxies can intercept the backend's response to modify headers, compress the payload (e.g., using Gzip), or even rewrite parts of the response body to ensure compatibility or security.
- **Caching:** After an API call completes, the reverse proxy can store the response. Subsequent requests for the same data are then served directly from the proxy's cache without hitting the backend again.
- **SSL/TLS Encryption:** A reverse proxy can handle the encryption of the response before it is sent back to the client, offloading this CPU-intensive task from the API server.
- **Response Filtering:** It can inspect responses for sensitive data (like leaked PII or internal server errors) and block or sanitize them before they reach the public internet.  Ngrok +6

Typical Sequential Flow

1. **Request:** Client → Reverse Proxy → API Backend.
2. **API Response:** API Backend → **Reverse Proxy**.
3. **Proxy Processing:** The proxy applies rules (compression, header injection, caching).
4. **Final Delivery:** Reverse Proxy → Client.  Ngrok +4

If your goal is to trigger a *new* action (like calling a different service) specifically *because* of an API response, you might be looking for **Middleware** or an **API Gateway**, which offer more advanced "post-processing" logic than a standard reverse proxy.